

Instrumentation & Optimization in Low-Latency C++ Applications

From Measurement to Mastery – Chapters 11 & 12

Reference: Building Low-Latency Applications with C++



Agenda

Overview & Structure

I.

Why Measure?

The case for instrumentation in low-latency systems

II.

Instrumentation Framework

RDTSC micro-benchmarking + TTT hop timestamps

III.

Performance Analysis

Jupyter workflows, visualizing distributions & latency spikes

IV.

Optimization Techniques

Logger (50x), MemPool (8x), Affinity, CRTP, Data Structures

V.

Future Extensions

Kernel bypass, FPGA acceleration, protocol enhancements

Why Instrumentation Matters

"You can't optimize what you can't measure"

- > **Challenge:** Nanosecond-scale timing requires extreme precision with minimal observer effect.
- > **Trade-off:** Every measurement instruction adds CPU overhead — implementation must be lightweight.
- > **Strategy:** Two complementary approaches are needed for complete system visibility.

RDTSC Micro-bench

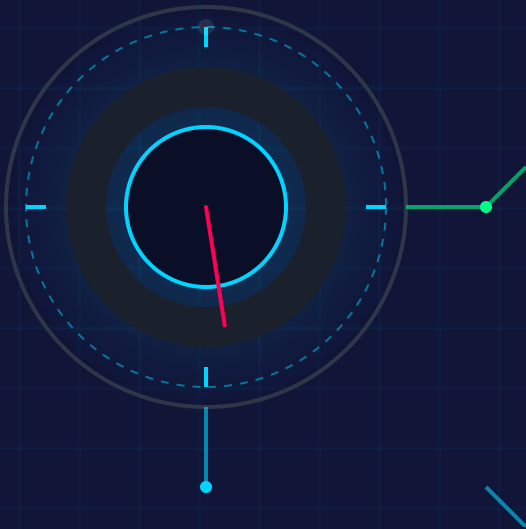
Measures CPU cycles for individual functions with minimal cost (~25-35 cycles).

HOT PATH PROFILING

TTT Hop-to-Hop

Absolute nanosecond timestamps tracking data transit across component boundaries.

END-TO-END LATENCY



RDTSC: Cycle-Accurate Micro-timing

The Time Stamp Counter (TSC) — a 64-bit CPU register counting clock cycles since reset

common/time_utils.h

```
1 // Read CPU Time Stamp Counter - minimal overhead
2 inline auto rdtsc() noexcept {
3     unsigned int lo, hi;
4     __asm__ __volatile__ (
5         "rdtsc" : "=a" (lo), "=d" (hi)
6     );
7     return ((uint64_t) hi << 32) | lo;
8 }
```

noexcept — zero exception overhead

__asm__ __volatile__ — prevents compiler reordering

Combines **hi:lo** into 64-bit value



~25-35 cycles
RDTSC OVERHEAD



~100+ cycles
STD::CHRONO::HIGH_RESOLUTION_CLOCK

Measurement Macros — RDTSC + TTT

Two complementary macro systems for different measurement granularities

common/macros.h — RDTSC Macros

```
#define START_MEASURE(TAG) \  
const auto TAG = Common::rdtsc()  
  
#define END_MEASURE(TAG, LOGGER) \  
do { \  
const auto end = Common::rdtsc(); \  
LOGGER.log("% RDTSC " #TAG " %\n", \  
Common::getCurrentTimeStr(&time_str_), \  
(end - TAG)); \  
} while(false)
```

Measures CPU cycles between START and END —
logs tag + delta cycles

common/macros.h — TTT Macro

```
#define TTT_MEASURE(TAG, LOGGER) \  
do { \  
const auto TAG = Common::getCurrentNanos(); \  
LOGGER.log("% TTT " #TAG " %\n", \  
Common::getCurrentTimeStr(&time_str_), \  
TAG); \  
} while(false)
```

Captures absolute nanosecond timestamp — for
tracking transit between system hops



High-Granularity Time Utilities



getCurrentNanos()

Returns absolute nanosecond timestamp using `std::chrono::system_clock::now()` and `duration_cast<nanoseconds>`



getCurrentTimeStr()

Formats time as HH:MM:SS.nnnnnnnnn with 9-digit nanosecond precision using `sprintf`



Why Both?

TTT uses `getCurrentNanos()` for absolute timestamps; `getCurrentTimeStr()` provides human-readable log entries for analysis

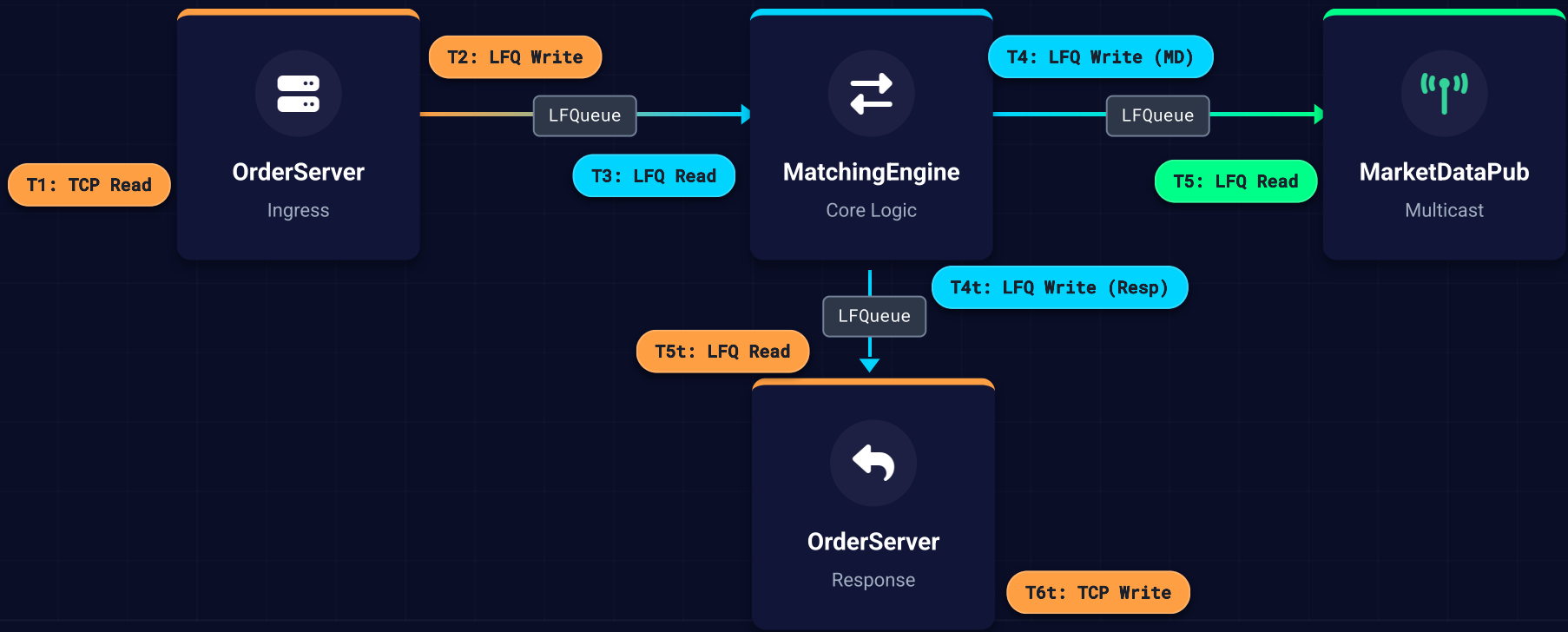
common/time_utils.h

```
// Nanosecond timestamp source
inline auto getCurrentNanos() noexcept {
    return std::chrono::duration_cast<
std::chrono::nanoseconds>(
std::chrono::system_clock::now()
.time_since_epoch()
).count();
}

// Human-readable time string
// Format: HH:MM:SS.nnnnnnnnn
inline auto getCurrentTimeStr(
std::string* time_str) {
    const auto clock =
std::chrono::system_clock::now();
    // ... sprintf with nanosecond precision
}
```

Exchange Topology & Timestamp Hops (T1–T6)

Tracking data flow through the exchange — from TCP ingress to UDP/TCP egress





Client Topology & Timestamp Hops (T7–T12)

Tracking data flow through the trading client – from UDP/TCP ingress to TCP egress



● Market Data Path (T7 → T9) ● Order Path (T10 → T12) ● Response Path (T7t → T9t)

Instrumenting Critical Functions

Where to place START/END_MEASURE for maximum insight

Exchange Side

- processClientRequest()
- addOrder() / removeOrder()
- match() / checkForMatch()
- sequenceAndPublish()
- send()

Client Side

- PositionKeeper::addFill()
- PositionKeeper::updateBBO()
- FeatureEngine::onOrderBookUpdate()
- RiskManager::checkPreTradeRisk()
- OrderManager::moveOrder()

```
exchange/me_order_book.cpp

auto MEOrderBook::add(ClientId id,
    OrderId oid, Side side,
    Price price, Qty qty) noexcept {

    START_MEASURE(Exchange_MEOrderBook_add);

    // ... order book logic ...
    const auto order = order_pool_.allocate(
        oid, side, price, qty, ...);
    addOrder(order);
    match();

    END_MEASURE(Exchange_MEOrderBook_add,
        logger_);
}
```

Pattern: Wrap the critical section — tag names follow Component_Class_Method convention

Performance Analysis Workflow

From raw logs to actionable insights – using Python & Jupyter

Parse Logs

Read TTT and RDTSC log lines. Extract timestamps, tags, and delta values. Filter by component.



Compute Metrics

Calculate hop-to-hop deltas (TTT). Convert RDTSC cycles to microseconds. Compute min, max, mean, p50, p99.



Visualize

Scatter plots: time vs latency per function. Histograms for distribution shape. Percentile bands for tail analysis.



Diagnose

Tight distributions → stable code path. Spiky distributions → contention or burst-driven.
Correlate spikes with system events.

```
In [1]: import pandas as pd
        df = parse_log('exchange.log')
```

```
In [2]: df['latency_us'] = df['cycles'] / cpu_freq
df.plot.scatter(x='time', y='latency_us')
```

```
Out[2]: <AxesSubplot:xlabel='time', ylabel='latency_us'>
```

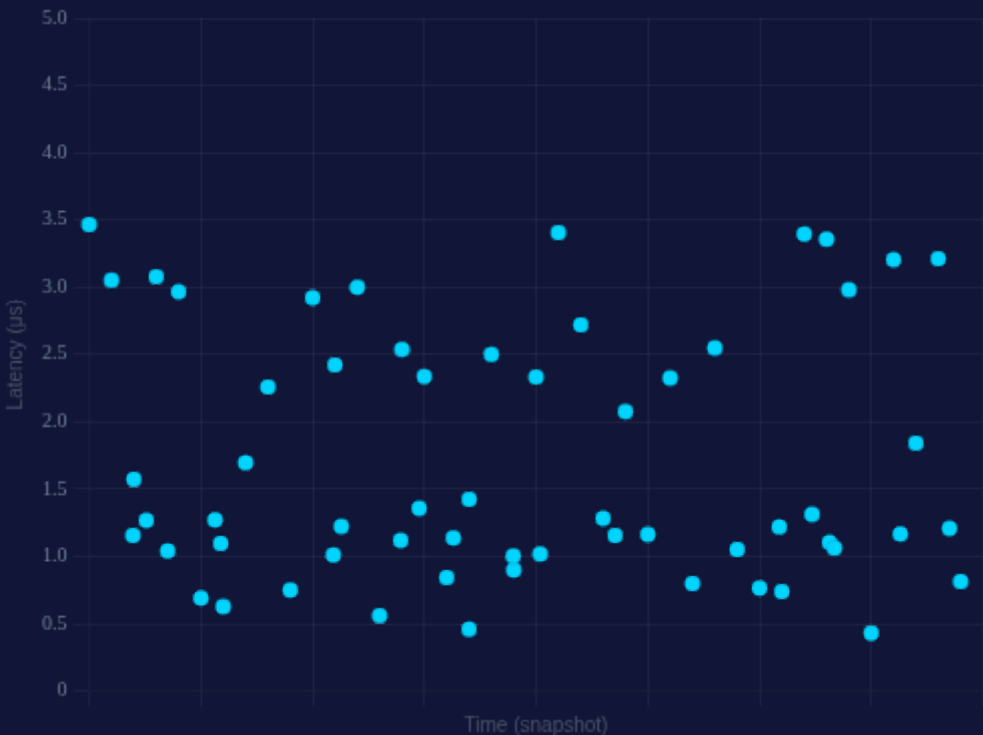
Latency Distributions — Exchange Components

RDTSC micro-benchmarks of hot-path functions (scatter plots: time vs latency)

MEOrderBook::removeOrder()

Order cancellation latency

✓ TIGHT & STABLE



MIN
0.4 µs

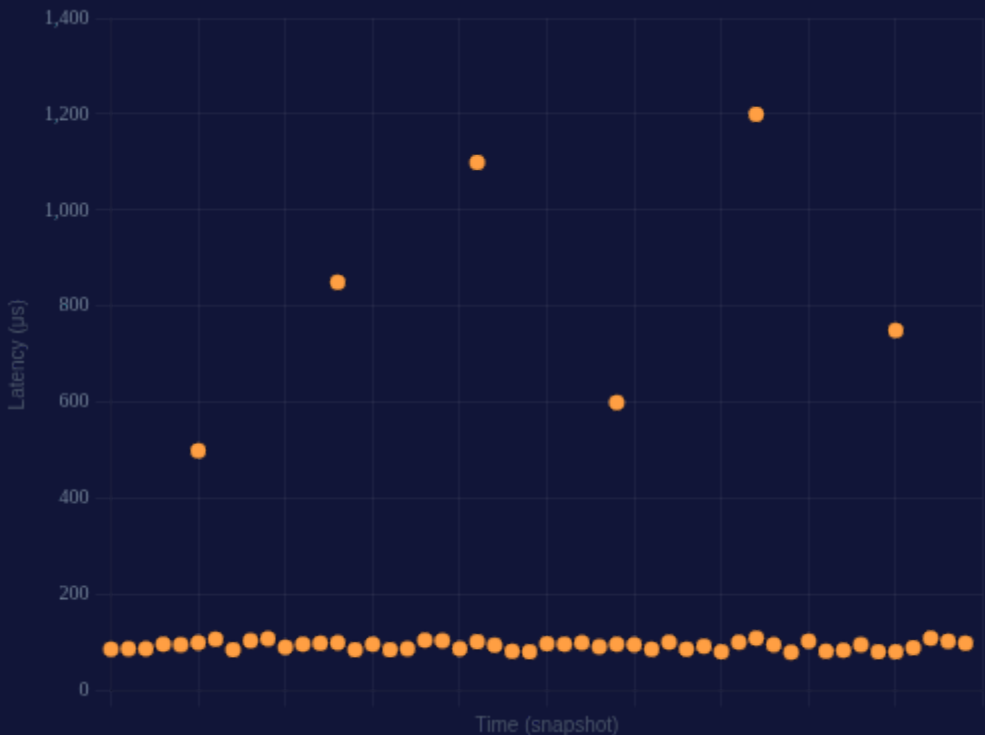
MEAN
1.2 µs

MAX
3.5 µs

FIFOSequencer::sequenceAndPublish()

Message sequencing & publication

⚠ SPIKY DISTRIBUTION





Code Demo — Instrumenting Exchange Hot Path

The C++ instrumentation behind slide 11's latency distributions

exchange/me_order_book.cpp

```
// MEOrderBook::removeOrder()
// Measured in slide 11 - left chart
auto MEOrderBook::removeOrder(
    ClientId c_id, OrderId o_id,
    Side side) noexcept {

    START_MEASURE(
        Exchange_MEOrderBook_removeOrder);

    auto& orders =
        (side == Side::BUY)
        ? bids_ : asks_;
    auto* order = id_map_[o_id];
    orders.erase(order);
    order_pool_.deallocate(order);

    END_MEASURE(
        Exchange_MEOrderBook_removeOrder,
        logger_);
}
```

exchange/fifo_sequencer.cpp

```
// FIFOSequencer::sequenceAndPublish()
// Measured in slide 11 - right chart
auto FIFOSequencer::
    sequenceAndPublish() noexcept {

    START_MEASURE(
        Exchange_FIFOSequencer_seqPub);

    // Sequence pending requests
    for (auto& req : pending_) {
        req.seq_num_ = next_seq++;
    }
    // Publish to matching engine
    for (auto& req : pending_) {
        me_queue_-->push(req);
    }
    pending_.clear();

    END_MEASURE(
        Exchange_FIFOSequencer_seqPub,
        logger_);
}
```

Sample RDTSC Log Output

```
15:28:42.123456789 RDTSC Exchange_MEOrderBook_removeOrder 26
15:28:42.123789012 RDTSC Exchange_FIFOSequencer_seqPub 205247
```

→ 26 cycles $\approx$ $\sim 1 \mu s$ (tight)

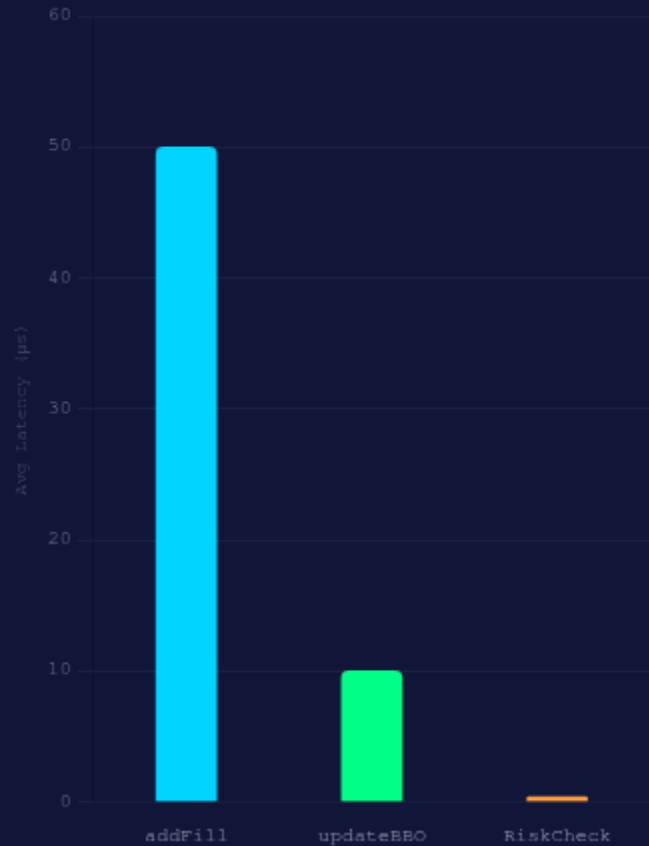
→ 205,247 cycles $\approx$ $\sim 90 \mu s$ (with spikes)

Latency Distributions — Client & Network Transit

Measuring client-side components and inter-system network latency

Client Components

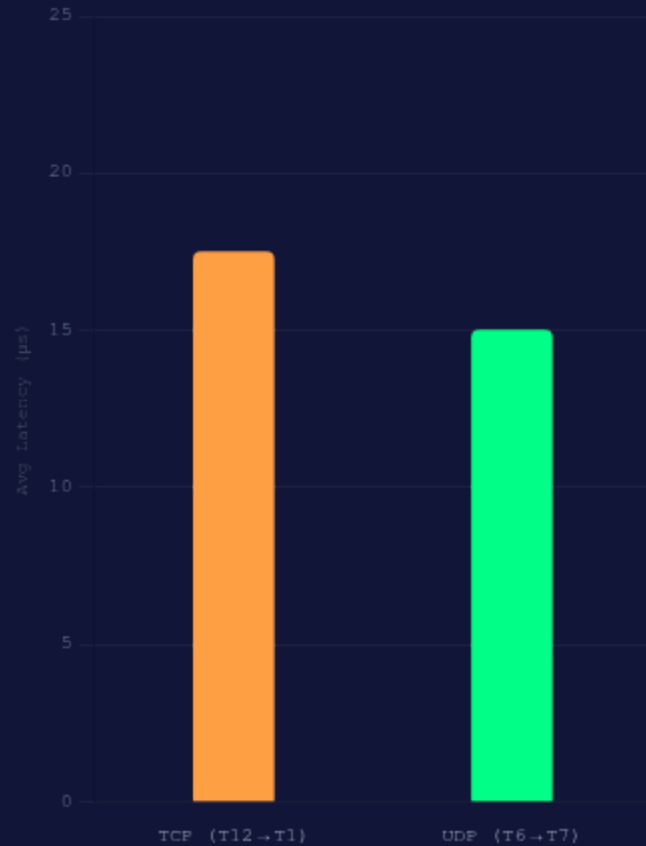
PositionKeeper & RiskManager



updateBBO is ~5× faster than addFill

Network Transit

Inter-system Latency (1 Hop)

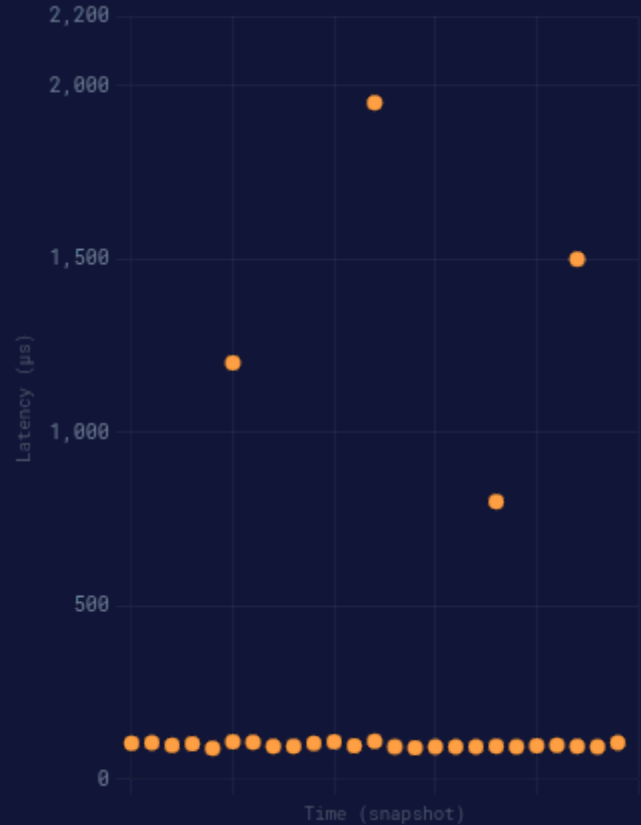


Network adds ~15-20 µs per hop

Consumer Queue (T7→T8)

⚠ TAIL LATENCY CONCERN

MDC to LFQueue Latency



AVG
~100 µs

SPIKES
2,000 µs



Optimization #1: Logger — 55× Faster

Eliminating char-by-char iteration on the hot path

✖ BEFORE

String values iterated character-by-character through LogElement queue

```
// Old: each char = one LogElement
for (auto c : str) {
    pushValue(c); // per-character!
}
```

25,757 cycles/op

AVERAGE LATENCY

✔ AFTER

Block copy via strncpy into fixed-size buffer in union — single push per string

```
enum class LogType : int8_t {
    // ... existing types ...
    STRING = 9
};
struct LogElement {
    LogType type_;
    union { char s[64]; /*...*/ } u_;
};
auto pushValue(const char* v) noexcept {
    LogElement l{LogType::STRING, {}};
    strncpy(l.u_.s, v, sizeof(l.u_.s)-1);
    pushValue(l);
}
```

466 cycles/op

AVERAGE LATENCY

55× SPEEDUP

Measured via RDTSC benchmark harness



Optimization #2: MemPool — 8× Faster

Removing safety checks from the hot path in release builds

✖ BEFORE

ASSERT() checks on every allocate() and deallocate() call — even in production

```
template<typename T>
T* MemPool<T>::allocate(T val) noexcept {
    ASSERT(next_free_ != nullptr,
           "MemPool exhausted");
    auto* obj = next_free_;
    ASSERT(obj->is_free_,
           "Expected free object");
    next_free_ = obj->next_free_;
    obj->is_free_ = false;
    // ...
}
```

343 cycles/op

AVERAGE LATENCY

✔ AFTER

Compile out ASSERT with NDEBUG preprocessor flag — zero runtime cost in release

```
// In CMakeLists.txt:
// target_compile_definitions(
// app PRIVATE NDEBUG)

#ifndef NDEBUG
#define ASSERT(cond, msg) \
    assert((cond) && (msg))
#else
#define ASSERT(cond, msg) ((void)0)
#endif
```

44 cycles/op

AVERAGE LATENCY

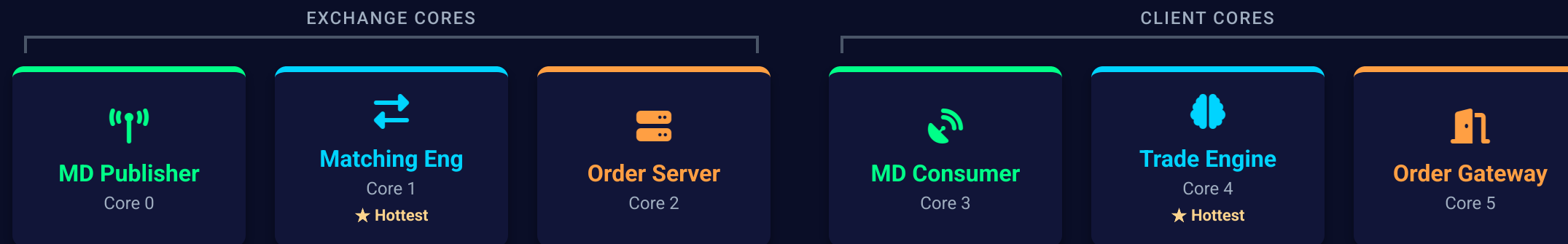
~8× SPEEDUP

Safety checks remain active in debug builds — best of both worlds



Optimization #3: Thread Affinity & CPU Isolation

Pinning critical threads to dedicated cores — eliminating OS scheduler interference



Thread Pinning API

```
Common::createAndStartThread(  
    -1, // affinity: core_id  
    "MatchingEngine",  
    [this]() { run(); }  
);
```

Linux Kernel Parameters

```
# /etc/default/grub  
GRUB_CMDLINE_LINUX="isolcpus=0-5  
nohz_full=0-5 rcu_nocbs=0-5"
```

Benefits: Zero context switches on isolated cores • Stable p99 latencies • Reduced cache pollution



Optimization #4: Eliminating std::function

Replacing runtime polymorphism with compile-time resolution via CRTP

✗ BEFORE — RUNTIME DISPATCH

std::function wraps callbacks with type erasure — heap allocation + virtual-like indirection

```
class OrderServer {  
    // Runtime dispatch - slow!  
    std::function<void(ClientRequest&)>  
    recv_callback_;  
  
    void onRecv(ClientRequest& req) {  
        recv_callback_(req); // indirect call  
    }  
};
```

- Heap allocation for captures
- Indirect function pointer call
- Branch predictor unfriendly

✓ AFTER — COMPILE-TIME CRTP

Templates + CRTP resolve dispatch at compile time — zero overhead, fully inlineable

```
template<typename Handler>  
class OrderServer {  
    void onRecv(ClientRequest& req) {  
        // Compile-time dispatch!  
        static_cast<Handler*>(this)  
        ->onClientRequest(req);  
    }  
};  
  
struct MyHandler : OrderServer<MyHandler> {  
    void onClientRequest(  
        ClientRequest& req) { /*fast*/ }  
};
```

- Zero heap allocation
- Direct inlined call
- Perfect branch prediction

CRTP: Curiously Recurring Template Pattern

Derived class passes itself as template parameter to base — enables static polymorphism



Optimization #5: Data Structure Tuning

Choosing the right hash map for MEOrderBook — fixed arrays vs dynamic maps

`std::array-based` Hash Map

Fixed-size, cache-friendly, zero allocation

142,650 cycles/op

✓ FASTEST FOR FIXED WORKLOADS

`std::unordered_map`

Dynamic growth, scalable, standard library

152,457 cycles/op

~6-7% OVERHEAD, BUT SCALES

Array Hash Map

Baseline (100%)

142,650

Unordered Map

+6.9% Overhead

152,457

ALTERNATIVE HASH MAPS TO EXPLORE

`absl::flat_hash_map`

Open addressing, flat memory layout

`robin_hood::map`

Robin Hood hashing, cache-efficient

`tsl::hopscotch_map`

Hopscotch hashing, good for small keys

`folly::AtomicHashMap`

Lock-free concurrent access (Meta)

`emhash7::HashMap`

Minimal overhead, benchmark champion

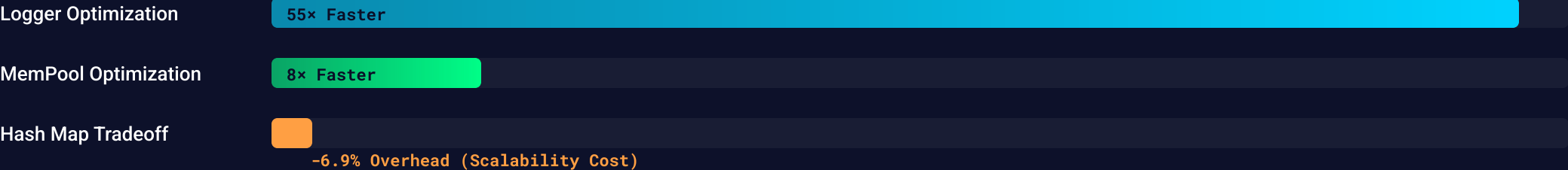


Benchmark Gains — Summary

Measured improvements across all optimization techniques

COMPONENT	BEFORE (CYCLES)	AFTER (CYCLES)	SPEEDUP	CATEGORY
Logger (string handling)	25,757	466	~55×	I/O PATH
MemPool (alloc/free)	343	44	~8×	MEMORY
std::function → CRTP	indirect	inlined	∞	DISPATCH
Thread Affinity	jittery p99	stable p99	—	OS TUNING
Hash Map (array → umap)	142,650	152,457	-6.9%	DATA STRUCT

RELATIVE SPEEDUP MAGNITUDE



Key Insight: The biggest performance gains came from eliminating unnecessary work on the hot path — not from algorithmic changes, but from removing overhead (char iteration, safety checks, indirection).



Future Extensions & Roadmap

Beyond the book — where to take the system next



Kernel Bypass I/O

DPDK, AF_XDP, RDMA — eliminate kernel overhead for network packets. Target: sub-5μs TCP transit



FPGA Acceleration

Offload matching engine logic to hardware. Deterministic sub-microsecond execution. No OS jitter.



MODIFY Messages

Add ClientRequestType::MODIFY — adjust price/qty without cancel-replace. Update OrderServer, ME, algos.



Snapshot Optimization

Spread snapshots per TickerId across time instead of bulk 60s bursts. Reduce network spikes & packet loss.



Dynamic Containers

Growable LFQueue & MemPool with fallback allocation. Handle rare overflow edge cases gracefully.



Richer Algorithms

Enhanced RiskManager metrics, complex OrderManager lifecycle, deeper FeatureEngine signals & compositions.



Key Takeaways

01

Measure Before Optimizing

Use RDTSC and TTT to establish baselines before changing any code

02

RDTSC for Micro, TTT for Macro

Combine cycle-level function profiling with nanosecond hop-to-hop transit tracking

03

Remove Unnecessary Work

Biggest wins: Logger 55× (block copy), MemPool 8× (remove asserts), eliminate std::function

04

OS-Level Tuning Stabilizes Tails

Thread affinity + isolcpus reduces jitter and tightens p99 latency distributions

05

Profile Continuously & Iterate

Performance is not a one-time fix — instrument, analyze, optimize, repeat



Q & A

Questions?